Formal Verification of Security Properties on RISC-V Processors

Czea Sie Chuah
c.chuah@eu.denso.com
DENSO AUTOMOTIVE Deutschland GmbH
Germany
Technical University of Munich
Germany

Christian Appold
Tim Leinmueller
c.appold@eu.denso.com
t.leinmueller@eu.denso.com
DENSO AUTOMOTIVE Deutschland GmbH
Germany

ABSTRACT

Hardware Security and trustworthiness are becoming ever more important, especially for security-critical applications like autonomous driving and service robots. With the increase in distribution of RISC-V processors, security issues in them arise. Security vulnerabilities and design flaws in processors can be exploited by attackers, e.g. by running software exploiting the vulnerabilities. This can lead to drastic consequences like damaging whole system functionality and even human lives can be endangered. Hence, it is very important to verify compliance of processors with the design specification and microarchitecture intent to harden the hardware against malicious attacks. Detection and removal of design bugs results in improved processor security. Therefore, we formally verify in this paper the security-critical functionality of a commercial RISC-V processor using model checking based formal verification with the formal verification tool Jasper. For this, we determined and implemented a comprehensive list of properties for security-critical functionality, derived from RISC-V specification and processor microarchitecture intent. The properties cover the security-critical functionality within a RISC-V processor. With our verification experiments, we detected design bugs which have been confirmed by the design team.

CCS CONCEPTS

• **Computer systems organization → System on a chip**; **Embedded systems**; • **Security and privacy → Logic and verification**; **Embedded systems security**; • **Hardware → Assertion checking**; **Model checking**.

KEYWORDS

Hardware Security, Formal Verification, Model Checking, Security-Critical Functionality, RISC-V, Processor

ACM Reference Format:
Czea Sie Chuah, Christian Appold, and Tim Leinmueller. 2023. Formal Verification of Security Properties on RISC-V Processors. In *21st ACM-IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE '23), September 21–22, 2023, Hamburg, Germany*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3610579.3611085

MEMOCODE '23, September 21–22, 2023, Hamburg, Germany

ACM ISBN 979-8-4007-0318-8/23/09…$15.00
https://doi.org/10.1145/3610579.3611085

1 INTRODUCTION

Processors are ubiquituous in our daily live and are embedded in nearly every electronic device. They enable a lot of very useful applications and due to this enrich and simplify our daily life. To function reliably and to fulfill their purpose, they have to be designed according to specification and to be free of bugs and design flaws. Otherwise, malfunctioning of processors can even endanger human life, especially in safety- and security-critical areas like autonomous driving or service robots. On the other hand more and more features, like pipelining, branch prediction or out-of-order execution are used in processors and due to this, processor complexity is continually increasing. As a consequence, processor design is getting more and more error-prone.

In the past years several famous processor bugs and vulnerabilities have been discovered. In the early 1990s the Pentium bug lead to a big loss of reputation for Intel [9]. Recently, Spectre and Meltdown have been published [14], which exploit security-vulnerabilities in processors to leak secret data. It is expected that even more security-vulnerabilities will be discovered and exploited in the near future. This unveils the underlying challenges in hardware security even though it had been a focus of research for the past decade. Several upcoming and promising new application areas like autonomous driving, IoT and factory automatization even have very high security demands. Hence, processors used in these applications have to fulfill very high security requirements and it is expected that the security challenges will continue to grow in future. Therefore, processor security is of utmost importance.

RISC-V instruction set architecture (ISA) is getting to be adopted more and more extensively on open source and commercial processor designs, due to its flexibility to support extensions and customization and its open instruction set. Forecasts predict that dispersion of RISC-V processors will continue to grow strongly in the coming years. Companies have taken the opportunity of the free ISA to design chips with high-end performance requirements supporting automotive and industrial applications [12, 16, 19]. The openness of the RISC-V ISA is a big advantage in terms of having a huge community to examine and extend the ISA on an ongoing basis, but at the same time this is also a disadvantage, as attackers can target the architecture easier. Due to the steadily growing dispersion and to enable the use of RISC-V processors in highly safety- and security critical areas, it is therefore inevitable to ensure high security standards and compliance to specification for RISC-V processors.

Security of a processor can be validated during pre-silicon and post-silicon phase and there are different methods for security checks with different purposes. Pre-silicon validation is done before silicon are being fabricated, thus changes and fixes on bugs can be carried out cost-efficiently. Post-silicon validation is executed with a fabricated hardware chip causing extremely high costs if any alteration is needed. Hence, vulnerabilities in processor designs should be identified as early as possible in the hardware life cycle. For this reason, we are focusing on detecting security vulnerabilities of processors in the pre-silicon phase by using formal verification in this paper.

Traditionally, simulation is widely used in the industry for pre-silicon validation since the dawn of processor evolution. However, the limitation of using this method is that verification is done only for stimulated input patterns, which results in corner case errors to be overseen. Due to growing processor complexity, it is getting even more difficult to investigate sufficient parts of the design functionality, because too many test cases need to be generated for it. Another method for pre-silicon hardware verification is formal verification. A fully-automatic approach for formal verification, which is used in this work is Model Checking. Model checking is a type of formal verification which uses a mathematical model of a system and systematically explores all possible behaviors of the system. There are previous works which target formal verification of processor security [6, 15]. Neither of these works targets RISC-V processors and for our work we did a much more comprehensive security investigation. As a result we are presenting in this paper a much more extensive set of security properties and our work is the first which enables detailed security-critical functionality formal verification for RISC-V processors. In the following we present in section II of the paper related work. Afterwards, section III continues with our used methodology, before section IV describes our security-critical properties. Section V discusses about the results of our formal verification work and finally section VI concludes the paper with direction on future work.

2 RELATED WORK

There has been formal verification work to verify against ISA compliance on ARM [18] and RISC-V architecture [21]. Regarding RISC-V there has been published a RISC-V Formal Verification Framework with a Formal Verification Interface (RVFI) [21] and any RISC-V processor which implements the RVFI can be formally verified for design compliance with the ISA using this framework. Both of those formal verification approaches prove the correctness of the design with respect to the ISA and detect corresponding design bugs in processors. The main focus of these frameworks is to verify that instructions work as desired according to specification. In comparison to this approach, our main focus is to target directly and exhaustively the security-critical functionality with our properties. Additionally, we target for full-proofs with our more directed properties instead of using Bounded Model Checking.

Papers [6, 15] analyse security properties on the OpenRISC OR1200 processor, with 13 security-critical properties identified and implemented respectively. Paper [10] formally verified physical memory protection (PMP) in an open source RISC-V core, Rocket Chip. Paper [8] presents work on security requirements using assertion checkers and code coverage method, with demonstration on OpenRISC processor. A runtime monitor evaluation against 18 security bugs on an OpenRISC OR1200 processor is presented in [13], while [7] define and determine security-critical properties with reference to the MIPS architectural specification for runtime monitoring. Our security-critical property set is much more comprehensive and our work is the first such work which targets the RISC-V architecture.

Both paper [7, 13] implement security-critical properties manually for runtime monitoring while [22] implemented a tool chain called SCIFinder workflow. They use machine learning to identify security-critical properties and as a result discover additional security-critical properties that were not discovered in previous manually written papers. They classify these security-critical properties finding into 5 categories, without considering a few other aspects which are security-critical that we have considered in our work.

There are only two other previous works [6, 15] which use formal verification for security properties verification. But they are done on an OpenRISC processor and additionally they are missing some aspects in comparison with our work and our property set is much more comprehensive. There is only one recent work which formally verified RISC-V but focusing only on PMP. We are the first who present a comprehensive set of security properties for RISC-V processors, as a result of a detailed security analysis. By doing this we fill an important gap in the existing literature about processor security verification. Our research work enables security-critical properties formal verification for RISC-V processors. We present our findings by classifying the security-critical properties into 9 categories which focus on different security aspects of a RISC-V processors.

3 METHODOLOGY

In this section we describe the methodology, which we used for our work. The used workflow is shown in Fig. 1. We start with studying specification documents and information about the microarchitecture of the processor to identify functionality which is security-critical [3–5, 20]. Afterwards, we derive and implement properties from the identified security-critical functionality and finally feed them into a commercial formal verification tool to be formally verified. Each step will be described in detail in the following subsections. Our experiment is done on the commercial automotive-grade RISC-V processor, Akaria NS31A which is given license from NSITEXE, Inc. [16]. NS31A is optimized for control-specific microcontroller. NS31A is a RISC-V processor, compliant to RISC-V instruction set architecture (ISA). It is a 32-bit single-issue processor, with a 4-stage pipeline with the pipeline stages *instruction fetch* (IF), *instruction decode* (ID), *execute* (EX) and *write back* (WB). The NS31A core supports 2 privilege modes: machine mode and user mode. It also comes with a branch prediction mechanism with a branch target buffer and a Floating Point Unit. Hardware thread (Hart) control finite state machine (FSM) is included in NS31A core and indicates hart execution states which are independent from the privilege mode of the hart to control state transition requests from

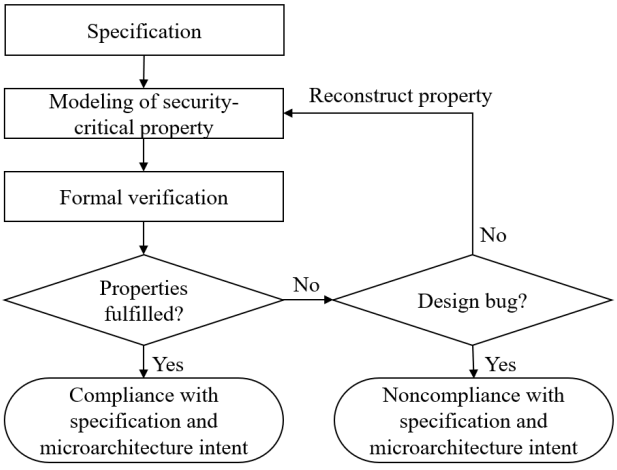

Figure 1: Methodology and Workflow

outside the core, debug control request, the execution of instructions, etc.

3.1 Specification

First, we deep dive into the RISC-V specification documents and NS31A microarchitecture implementation. In-depth knowledge of the specification documents and the microarchitecture is required to derive information on security-critical functionalities inside the processor. In order to achieve this, we run through numerous literature, the RISC-V unprivileged [4] and privileged [5] instruction set manual, RISC-V debug specification [20] together with the NS31A technical reference manual (TRM) [3]. The unprivileged instruction set manual describes the RISC-V unprivileged architecture. It includes the RISC-V base integer ISA, that must be present in any implementation, with optional extensions to the base ISA. The privileged instruction set manual on the other hand covers all aspects of RISC-V systems beyond the unprivileged ISA, including privileged instructions as well as additional functionality required for running operating systems and attaching external devices. The debug specification outlines debug support for the RISC-V platform. The TRM documents hardware configuration, logic specification and implementation details of the NS31A processor. In our work, we target to cover the security-critical functionalities within the RISC-V core comprehensively.

3.2 Modeling of security-critical properties

After identifying security-critical functionalities of the processor in specification and microarchitecture intent, we formulate the requirements on the correct processor behavior in the form of suitable properties. For example, mode transition from user mode to machine mode is allowed to happen only when it is admitted, e.g. when an exception or interrupt is executed. Properties of these security-critical functionality are formulated as SystemVerilog Assertions (SVA) [2] using concurrent assertions as shown below. SVA is a powerful language to model constraints, rules and behaviour for validating correctness of the design.

Concurrent SVA:

```
assertion_name: assert property (@(posedge clk)
disable iff (rst) (x && !y) |-> ##2 (z == $past(z)));
```

- `assertion_name` : label of the assertion expression to be described, usually with an obvious naming of the behaviour.
- `assert property` : syntax of a concurrent assertion, without '`property`' the assertion is immediate.
- `@(posedge clk)` : the property is sampled at the positive edge of the signal '`clk`'.
- `disable iff (rst)` : the property is disabled when the '`rst`' signal is high.
- `(x && !y)` : antecedent, which means the pre-condition of the property to be fulfilled, this can be a combination of a set of signals or just a signal by itself.
- `|-> ##2` : the expression is checked after the number of clock cycles specified (in this case 2) after pre-condition has been fulfilled.
- `z == $past(z)` : consequent, which is the expression behaviour to be checked, such as constant value, rise or fall of a signal, etc. In this example, comparison with a previous value (`$past()`) is done.

In the following when properties are listed, we omit the first half of this SVA example and start immediately with the formula of the property due to space constraint.

3.3 Formal verification and analysis

Thirdly, the assertion properties together with the design are fed into the Cadence Jasper Formal Verification Platform [17] for validation. Jasper can generate cover property automatically for implication properties on the pre-condition of the implication. If the cover property of an implication property passes, this means the pre-condition has been triggered during the verification. If the assertion passes, the property is fulfilled by the design. When the implication cover fails, it means that the pre-condition cannot be fulfilled for any behavior in the design. In this case, the implication consequent has not been checked at all during verification. If the assertion fails, it means that the property is violated and a corresponding counter example will be generated by the tool. The counterexample shows a design behavior where the property fails and this can be analyzed to determine whether the behavior is caused by a design bug or an incorrect properties modeling. If the failure is caused by incorrect properties modeling, we modified the property and repeat the previous steps. On the other hand, if the failure is not caused by incorrect properties modeling, then it would be an noncompliance to the specification or microarchitecture intent that needs to be fixed.

The advantage of using formal verification is its exhaustiveness as it investigates all design behaviors for validity of the specified properties. Thus it can also detect bugs appearing only in rare corner cases. This is a huge advantage in comparison to simulation method, where corner cases are often hard to stimulate. The application of formal verification is a big improvement especially for security verification. Design behaviors that are not proven by simulation could lead to security vulnerabilities, whereas formal verification can reliably detect security vulnerabilities for all design

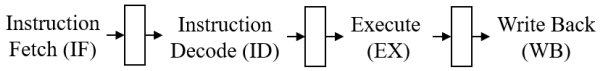

Figure 2: Pipeline of NS31A RISC-V Processor

behaviors. Model Checking based formal verification can also be applied relatively quickly in early design stages, as no test benches have to be written like for simulation. It is reliable since it provides a systematic and automated way to verify system requirements with regards to the modelled properties. Formal verification has been recommended in many safety and security standards to assure high level of quality assurance of a system. The Common Criteria (CC) for Information Technology Security Evaluation [1] provide an international standard for computer security certification. Its highest assurance levels EAL5 onwards require formal specification and verification methods[11].

4 SECURITY-CRITICAL FUNCTIONALITY

We have identified and verified in total 1146 security-critical functionality properties for the RISC-V core. We categorised them into 9 groups, namely instruction execution, control and status register (CSR), debug operation, exception and interrupt handling, mode transition, physical memory protection (PMP), control flow, register update and memory access. In the following subsections, we present and explain our properties.

4.1 Instruction Execution

NS31A is a 32-bit processor core with a 4-stage pipeline as shown in Figure 2. The 4 stages are *IF*, *ID*, *EX* and *WB*. This allows the processor to perform an instruction in multiple steps in an overlapping manner. The properties presented in this subsection prove proper instruction movement in processor and that ALU flags are set in the right way.

(1) *Property 1-2*: Instruction moving from one pipeline stage to another, which means from *ID* to *EX* (example below) and from *EX* to *WB* should remain the same.

```
inst_not_changed_id_to_ex: assert property
(@(posedge clk) disable iff (rst) idInstValid &&
idPipeReady |-> ##1 exInstCode==$past(idInstCode));
```

Note: "assertion_name: assert property (@(posedge clk) disable iff (rst)" will not be shown in succeeding properties of the paper due to page limit.

(2) *Property 3-4*: Instruction that stays within a pipeline stage should remain the same (example for *wb*).

```
wbInstValid && !wbPipeReady |-> ##1
wbInstCode == $past(wbInstCode)
```

(3) *Property 5-10*: ALU flag which influence the control flow should be set correctly for the following instructions: *beq* (example), *bne*, *blt*, *bge*, *bltu*, *bgeu*.

```
InstValid && PipeReady && beqOpcode && beqFunct3
&& Src1Data == Src2Data |-> equalFlag
```

4.2 Control and Status Register

CSRs are register for control and status acquisition that associate to a unique address space, and dedicated access instructions are provided. Each of these registers have its own functionality and usually correspond to a particular privilege level be it supervisor level, hypervisor level, or machine level providing protection between different software components. To access these CSRs, there are restrictions and rules to obey. Each CSR has their own read or write indication and the lowest privilege level that is allowed to access them. For instance, `mstatus` is a CSR for acquiring and controlling the current operating status of the hart in machine mode. Under this category, we implemented properties for all the 67 CSRs (when applicable) in the NS31A processor core with reference to the specification [5], technical reference manual [3] and Table 1. In Table 1, for first letter of CSR name, the symbol `f` floating point, `m` indicates machine mode, `t` indicates trigger and `d` indicates debug.

(1) *Property 11-111*: CSRs do not have write access in user mode except for user floating point CSRs (example for `mstatus`).

```
UserMode && CsrWriteOp |-> !(WriteEn_mstatus)
```

(2) *Property 112-162*: CSRs can only be written by instructions with matching CSR address (example for `mstatus`).

```
CsrWtAddr!=MstatusAddr |-> !(WriteEn_mstatus)
```

(3) *Property 163-228*: CSR can only be read by instructions with matching CSR address (example for `mstatus`).

```
CsrRdAddr!=MstatusAddr |-> RdData_mstatus==32'd0
```

(4) *Property 229-285*: CSR read access is done only for CSR instructions with corresponding register and immediate operand as shown in Table 2 (example for `mstatus`).

```
(!(CSRRW) && !(CSRRS) && !(CSRRC) && !(CSRRWI) &&
!(CSRRSI) && !(CSRRCI)) |-> CsrRdAddr!= MstatusAddr
```

(5) *Property 286-342*: CSR write access is done only for CSR instructions with corresponding register and immediate operand as shown in Table 2 (example for `mstatus`).

```
((!(CSRRW) && !(CSRRS) && !(CSRRC) && !(CSRRWI) &&
!(CSRRSI) && !(CSRRCI))) |-> !(WriteEn_mstatus)
```

(6) *Property 343-352*: Read-only CSR's value is constant as it should not be modified in any case (example for `mvendorid`).

```
CsrRdAddr==mvendoridAddr |-> CsrRdData==VendorID
```

(7) *Property 353-398*: CSR should be in a valid format. Some read/write CSR fields are only defined for a subset of bit encodings, but allow any value to be written while guaranteeing to return a legal value whenever read [5]. As shown in below example, reserved bits in `mie` should be `0`.

```
{MieRegOut[31:20],MieRegOut[18:12],MieRegOut[10:
8],MieRegOut[6:4],MieRegOut[2:0]} == 23'd0
```

(8) *Property 399-404*: CSR access violation are checked as follow:

4.2.1 Property 399-400. Exception flag is set for write to a read-only CSR register.

4.2.2 Property 401-402. Exception flag is set for attempts to access a non-existent CSR.

4.2.3 Property 403-404. Exception flag is set for attempts to access a CSR without appropriate privilege level.

Table 1: NS31A Control and Status Register [5][3]

Description	Access Mode	
Register	Read	Write
User Floating-Point CSRs		
`fflags` Accrued exceptions	DMU	DMU
`frm` Dynamic rounding mode	DMU	DMU
`fcsr` Control and status register	DMU	DMU
Machine Trap Setup		
`mstatus` Status register	DM	DM
`misa` ISA and extensions	DM	-
`mie` Interrupt enable register	DM	DM
`mtvec` Trap handler base address	DM	DM
`mcounteren` Counter enable register	DM	DM
Machine Counter Setup		
`mcountinhibit` Counter-inhibit register	DM	DM
Machine Trap Handling		
`mscratch` Trap handler's scratch register	DM	DM
`mepc` Exception PC register	DM	DM
`mcause` Trap cause register	DM	DM
`mtval` Bad address or instruction	DM	DM
`mip` Interrupt pending register	DM	-
Machine Memory Protection		
`pmpcgf0-3` PMP configuration	DM	DM
`pmpaddr0-15` PMP address register	DM	DM
Debug/Trace Registers		
`tselect` Debug/trace select	DM	DM
`tdata1` First debug/trace register	DM	DM
`tdata2` Second debug/trace register	DM	DM
`tinfo` Information register	DM	-
Debug Mode Registers		
`mcountinhibit` Counter-inhibit register	D	D
`dcsr` Control and status register	D	D
`dpc` Program counter (PC) register	D	D
`dscratch0` Scratch register0	D	D
Exception Cause Flags for Access Fault*		
`meecaddr` Captured error address register	DM	DM
`meecunlock` Unlock control for `meecaddr`	DM	DM
Machine Counter/Timers		
`mcycle` Cycle counter	DM	DM
`minstret` Instruction retired counter	DM	DM
`mcycleh` Upper-32bits of `mcycle`	DM	DM
`minstreth` Upper-32bits of `minstret`	DM	DM
Hart/Cache Control*		
`mhtpc` Hart PC register	DM	DM
`mhtenable` Hart enable control status register	DM	DM
`mhtstatus` Hart run control status register	DM	DM
L1 Instruction Cache Control*		
`ml1cachesysctrl` L1 System Control	DM	DM
`ml1cacheoperation` L1 Operation	DM	DM
`ml1cacheprefcfg` L1 Instruction prefetch control	DM	DM
`mbpsysctrl` Branch prediction system control	DM	DM
`mbpoperation` Branch prediction operation	DM	DM
Hart Fetch address Register*		
`mhtfetchaddr` Hart fetch address register	DM	-
User Counter/Timers		
`cycle` RDCYCLE cycle counter	DMU	-
`time` RDTIME timer	DMU	-
`instret` RDINSTRET instructions-retired counter	DMU	-
`cycleh` Upper-32bits of `cycle`	DMU	-
`timeh` Upper-32bits of `time`	DMU	-
`instreth` Upper-32bits of `instret`	DMU	-
Machine Information Registers		
`mvendorif` Vendor ID	DM	-
`marchid` Architecture ID	DM	-
`mimpid` Implementation ID	DM	-
`mhartid` Hardware thread ID	DM	-

D: debug mode; M: machine mode; U: user mode
*NS31A extended CSR

Table 2: CSR read and write operand behaviour[4]

Register operand				
Instruction	rd	rs1	read CSR?	write CSR?
CSRRW	x0	-	no	yes
CSRRW	!x0	-	yes	yes
CSRRS/C	-	x0	yes	no
CSRRS/C	-	!x0	yes	yes
Immediate operand				
Instruction	rd	uimm	read CSR?	write CSR?
CSRRWI	x0	-	no	yes
CSRRWI	!x0	-	yes	yes
CSRRS/CI	-	0	yes	no
CSRRS/CI	-	!0	yes	yes

4.3 Debug Operation

Debug mode reserves a few CSR addresses that can only be accessed in debug mode. Since debug mode has the highest priority (even more than machine mode [5]), access permission for these debug CSRs are security-critical.

(1) *Property 405-407*:`dpc` settings should match the specification [20].
   (a) *Property 405* Upon entry to debug mode, `dpc` is updated with the address of the next instruction to be executed. *ebreak* sets `dpc` to address of *ebreak* instruction. The *ebreak* instruction is used to return control to a debugging environment.
   ```
   InstValid && PipeReady && EbreakDebug |-> ##1
   Dpc == $past(InstPC)
   ```
   (b) *Property 406-407* `dpc` is set to the address of the instruction that would be executed next if no debugging was going on where `DebugStepExe` is asserted i.e. `PC+4` for 32-bit instruction that does not change program flow, or the destination PC on taken jump/branch.
   ```
   InstValid && PipeReady && !EbreakDebug &&
   DebugStepExe && (JumpRequest || (BrExec &&
   BrTaken)) |-> ##1 Dpc == $past(JmpTarget)
   ```

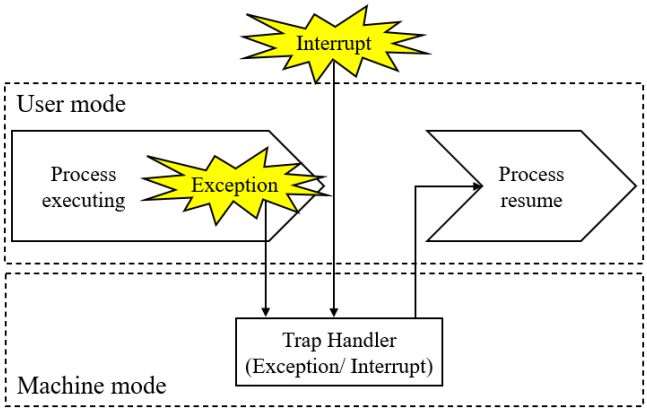

Figure 3: Illustration of Exception, Interrupt and Trap

(2) *Property 408*: `dcsr` should contain privilege mode that hart was operating when debug mode was entered [20].

```
EnterDebugMode |-> ##1
(Dcsr[1:0] == $past(PrivMode)
```

(3) *Property 409-418*: Write to CSR in mode other than debug mode is not allowed for debug mode register.

```
!DebugMode |-> !(WriteEn_dcsr)
```

4.4 Exception, Interrupt and Trap Handling

Exception and interrupt are unanticipated events that break the usual flow of instruction execution as shown in Figure 3. Exception is an unusual condition occurring internally within the processor while interrupt is an unexpected event that happens. Trap handler here refers to the handler which are called by either an exception or an interrupt. These events in the processor need to be handled properly to prevent disruption of continuous flow for the subsequent instructions. They also have prioritization according to the hardware specification. Table 3 shows the list of exception, interrupt and non-masking interrupt with decreasing priority.

(1) *Property 419-439*: Corresponding exception flag is set for all exceptions, interrupts and NMI in Table 3.
(2) *Property 440-482*: `mtval` and `mcause` are set to a value indicating the corresponding exception/interrupt [3].
  (a) Property 440-466: `mtval` is written with exception-specific information to assist software in handling the trap, corresponding to the exception, interrupt or NMI (example for Breakpoint).
  `InstValid && PipeReady && !DebugMode && Exception && ExcpFlag[19] |-> ##1 RegData_mtval == 32'd0`
  (b) *Property 467-482*: `mcause` is set to the exception code corresponding to the exception, interrupt or NMI (example for Breakpoint).
  `InstValid && PipeReady && !DebugMode && Exception && ExcpFlag[19] |-> ##1 RegData_mcause == 32'h3`
(3) *Property 483-490*: Exception and interrupt update status correctly [5].

Table 3: NS31A Exception and Interrupt [4]

Category	Description
Async	Reset
NMI	
Async NMI	External NMI Error NMI Machine non-blocking store bus error NMI Machine non-blocking load bus error NMI
Interrupt	
Async Interrupt	Error Interrupt External Interrupt Software Interrupt Timer Interrupt
Exception	
Instruction Fetch	Break (Instruction address) Instruction address misaligned Instruction access fault (PMP error) Instruction access fault (Cache error) Instruction access fault (Bus error)
Instruction Decode	Illegal instruction (Reserved instruction) Break (ebreak) Environment call from user mode Environment call from machine mode
Instruction Execution	Illegal instruction (Privilege error) Illegal instruction (CSR decode error)
Load/Store	Break (Load/Store address) Load address misaligned Store/AMO address misaligned Load access fault (PMP error) Store/AMO access fault (PMP error)

  (a) *Property 483-484*: When a trap is taken, previous privilege level for machine mode, `mpp` bit of `mstatus` is set to privilege level before exception or interrupt;
  (b) *Property 485-486*: Interrupt enable bit for machine mode, `mie` set to 0 when a trap is taken;
  (c) *Property 487-488*: Previous interrupt enable bit for machine mode, `mpie` set to value of previous `mie`;
  (d) *Property 489-490*: For exception, exception program counter for machine mode, `mepc` set to PC from *WB* stage while for interrupt, `mepc` has to be set to current PC value.
(4) *Property 491-498*: Exception and interrupt return update status correctly with *MRET* instruction. An *MRET* instruction is used to return from a trap in machine mode [3].
  (a) *Property 491-494*: By executing *MRET* when `mpp`=0, modify privilege bit `mprv`=0 and `mpp`=0.
  (b) *Property 495-496*: "1" is set to `mpie` when the state is returned to the previous state before the occurrence of a trap due to the *MRET* instruction.
  (c) *Property 497-498*: The value held in `mpie` is restored in `mie` when the state is returned to the previous state before the occurrence of a trap due to the *MRET* instruction.
(5) *Property 499-500*: Exception implies handled is verified with the following properties:

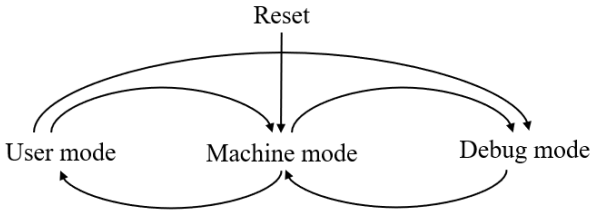

Figure 4: Mode transition

(a) *Property 499*: If an exception appeared exception handler should be started, if conditions for exception start are fulfilled.
(b) *Property 500*: Exceptions are not being handled in absence of any request for exception.

(6) *Property 501*: Interrupt implies handled. When interrupt happens, `interrupt_counter` increase by 1 and eventually interrupt service routine occur. `interrupt_counter` is decreased for every interrupt service routine start.

```
interrupt_counter!=6'd0 |-> strong
(##[0:$] InstPC == InterruptTrapEntry)
```

(7) *Property 502*: Interrupt is disabled/enabled according to value of `mie` bit[5]. Interrupt taken is enabled when `mie` bit is high and disabled when `mie` bit is low.

```
(|IntCause[31:3]) |-> IntTaken == MstatusMIE
```

(8) *Property 503*: The Interrupt bit in `mcause` is set if the trap was caused by an interrupt[5].

```
IntBrReq |-> ##1 RegData_mcauseINT
```

(9) *Property 504*: `mepc` register is written with the address of the instruction that was interrupted[5].

```
IntBrReq |-> ##1 RegData_mepc == $past(InstPC)
```

(10) *Property 505*: If frm is set to an invalid value (101–111), any subsequent attempt to execute a floating-point operation with a dynamic rounding mode is illegal.

4.5 Mode Transition

NS31A supports 2 privilege levels: user mode and machine mode. Machine mode is mandatory for a RISC-V processor to allow low-level access to the implementation. User mode is used for conventional application. In NS31A, debug mode is implemented as an additional privilege mode with further access than the machine mode which is used for off-chip debugging and manufacturing test. Transition between modes [4] can only occur under specific conditions and requirements. We formally verified correctness and validity of transition between the privilege modes as follow:

(1) *Property 506-507*: Transition into user mode valid for the following:
  (a) *Property 506*: From machine mode when *MRET* is executed.
  (b) *Property 507*: From debug mode after debug retires if user mode is stored in `dcsr` before entering debug.
(2) *Property 508-515*: Transition into machine mode valid for:
  (a) *Property 508*: From user mode when hart state allows.
  (b) *Property 509-510*: From debug mode immediately after debug retires.

```
DebugMode == 1'b1 ##1 DebugMode == 1'b0 |->
PrivMode == MachineMode
```

  (c) *Property 511*: During processor reset regardless of any circumstances. The property below is proven during reset, which is different from the other properties.

```
PrivMode == MachineMode && !MstatusMIE &&
MstatusMPP == UserMode
```

  (d) *Property 512*: When exception happens, privilege mode is set to machine mode.

```
exception_happen |-> ##1 PrivMode == MachineMode
```

  (e) *Property 513-515*: When interrupt happen, privilege mode is set to machine mode under the conditions below[5]:
    (i) either the current privilege mode is M and the mie bit in the `mstatus` register is set, or the current privilege mode has less privilege than machine mode;
    (ii) bit i is set in both mip and mie. mip bit high means a pending interrupt.

```
((MachineMode && Mstatus_Mie) && ((Mip[3] &&
Mie[3]) || (Mip[7] && Mie[7]) || (Mip[11] &&
Mie[11]) || (Mip[19] && Mie[19]))) |-> ##1
PrivMode == MachineMode
```

(3) *Property 516-517* Transition into debug mode valid for:
  (a) *Property 516*: From user mode in case of an halt request and when the hart control FSM permits it, change to debug mode is done. Halt request is a request to halt execution of a hart.
  (b) *Property 517*: From machine mode in case of a halt request and when the hart control FSM permits it, change to debug mode is done.

(4) *Property 518*: When return from debug/machine mode, *MRET* update privilege mode with the previous mode saved.

```
mret_occured |-> ##1 PrivMode == $past(MstatusMPP)
```

4.6 Physical Memory Protection

PMP acts as an access control for physical memory region within the RISC-V processor for instruction and data access. It also detects illegal access to any out-of-range address. Each PMP entry consists of an address register (`pmpaddr`) and a configuration (`pmpcfg`) associated with it as shown in Figure 5. The PMP entry 0 has the highest priority for determining the access and the priority decreases as the number gets higher. The functional correctness of PMP is vital to provide security guarantee on a RISC-V processor avoiding unwanted access to the memory or modification to the access permission.

`pmpaddr` stores the address registers which defines the memory range of the PMP entry and these address range is determined by the addressing mode:

- NA4 - Naturally aligned four-byte region
- NAPOT - Naturally aligned power-of-two region
- TOR - Top of range

`pmpcfg` consists of 8 bits each and 4 of these configuration are combined to be stored as one CSR:

- L - Lock bit indicates that the PMP entry is locked and configuration to the associated address registers are ignored.
- A - Addressing modes are described on the paragraph above.

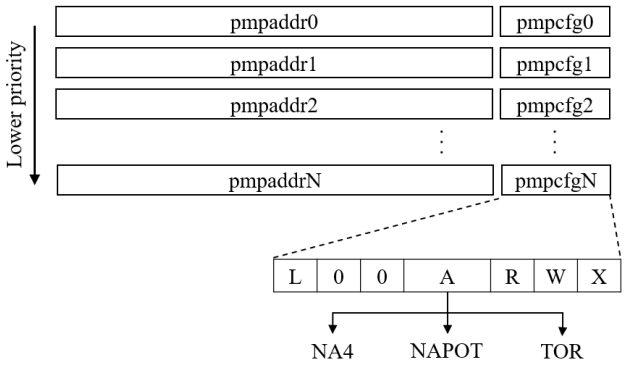

Figure 5: PMP address and configuration

- R - Read bit control the read access permission of the PMP entry.
- W - Write bit control the write access permission of the PMP entry.
- X - Execute bit control the execute access permission of the PMP entry.

(1) *Property 519-646*: PMP locking functionality correctness is checked with regards to the properties below:
   (a) *Property 519-550*: When PMP entry is locked, i.e., writes to the configuration register and associated address registers are ignored (example for `pmpcfg0`).
   `RegData_Pmpcfg0.L |-> !pmpaddr0_WtVld`
   (b) *Property 551-566*: Locked PMP entries remain locked until the hart is reset (example for `pmpcfg0`).
   `RegData_Pmpcfg0.L |->`
   `RegData_Pmpcfg0.L throughout !hartRst`
   (c) *Property 567-582*: If PMP entry N is locked, writes to `pmpcfgN` and `pmpaddrN` are ignored (example for `pmpcfg0`).
   `RegData_Pmpcfg0.L |-> ##1 $stable`
   `(RegData_PmpcfgRegs0[4:0])`
   (d) *Property 583-598*: If PMP entry N is locked and `pmpcfgN.A` is set to TOR, writes to `pmpaddrN-1` are ignored (example for `pmpcfg1`).
   `RegData_Pmpcfg1.L && RegData_PmpcfgRegs1.A==TOR`
   `|-> !pmpaddr0_WtVld`
   (e) *Property 599-630*: For NAPOT, writing to `pmpaddrN` is valid if `pmpcfgN.L=0` (example for `pmpaddr0`).
   `!RegData_Pmpcfg0.L && RegData_Pmpcfg0.A==NAPOT`
   `&& RegData_Pmpcfg1.A!=TOR && CSRWrite`
   `|-> pmpaddr0_WtVld`
   (f) *Property 631-646*: For TOR, writing to `pmpaddrN-1` and `pmpaddrN` is valid if `pmpcfgN.L=0` (example for `pmpcfg1`).
   `RegData_Pmp1.L && RegData_Pmpcfg1.A==TOR &&`
   `CSRWrite |-> !pmpaddr0_WtVld && !pmpaddr1_WtVld`

(2) *Property 647-1078*: PMP prioritization correctness for each addressing mode and permission access combination are checked here. If a PMP entry matches all bytes of an access, then the L, R, W, and X bits determine whether the access succeeds or fails. If it falls in more than one region, it matches the permission in the lowest region.

(3) *Property 1079-1084*: PMP access correctness through machine mode and user mode are checked with the properties below:
   (a) *Property 1079-1081*: If no PMP entry matches an M-mode access, the access succeeds. We check that the PMP entry does not fall under TOR, NA4 or NAPOT region.
   `Opcode==Pmp && !tor_in_region && !na4_in_region`
   `&& !napot_in_region && LSPrivMode==MachineMode`
   `&& !DebugMode |-> !LoadAccessFault`
   (b) *Property 1082-1084*: If no PMP entry matches an S-mode or U-mode access, but at least one PMP entry is implemented, the access fails. If at least one PMP entry is implemented, but all PMP entries' A fields are set to OFF, then all S-mode and U-mode memory accesses will fail.

(4) *Property 1085-1092*: PMP violation - If there is PMP violation (load access fault or store access fault), then for load there should be no read on the target register and for store there should be no write on the target register.

```
PipeRegEn && LoadAccessFault |-> ##1
RegData == $past(RegData)
```

4.7 Control Flow

It is very important that the PC is always set properly by the processor. Otherwise, wrong instructions could be fetched and executed, which could then be exploited by attackers to execute malicious code. We developed a new approach to formally verify the proper setting of the PC. Our PC properties evaluate the PC for each setting of the PC for a valid instruction into the pipeline register entering *WB* stage. With this, we formally verify the PC setting for each instruction that is executed in the processor and as a result we prove that the PC is always set correctly as implied on the specification. In addition to the properties for PC verification, we use some auxiliary code. If a valid instruction enters the *WB* stage that sets the PC to a value other than PC +4, we save this in the verification environment (e.g. for *jal* or executed branches). Besides that, we also store all events which happen between a valid instruction entering *WB* stage and the next valid instruction entering *WB* stage, e.g. interrupts. This information is then used to decide the correct PC for the successor instruction of a valid instruction entering *WB* stage according to RISC-V specification. With the properties listed in the following, we then prove the correctness for setting the PC. The properties below cover all possibilities of instructions and events that can happen which would influence the setting of the PC.

(1) *Property 1093*: For first executed instruction, PC has to be PC that should occur after reset.

```
firstInstruction && write_wb_pipe_reg &&
!interrupt_appeared |-> wb_InstPC == resetPC
```

(2) *Property 1094*: If previous instruction does not require other PC adaption and no special event appeared in between, standard PC increment occur.

```
!firstInstruction && write_wb_pipe_reg &&
!interrupt_appeared && !exception_appeared &&
!last_jal && !last_jalr && !last_mret &&
!last_branch_taken && !debug_return
|-> wb_InstPC == lastWbPC +4
```

(3) *Property 1095*: Prove for correct PC set when an interrupt occurred.

```
write_wb_pipe_reg && interrupt_appeared
|-> wb_InstPC == interruptEntry
```

(4) *Property 1096*: Prove for correct PC set when an exception occurred.

```
write_wb_pipe_reg && exception_appeared
|-> wb_InstPC == exceptionEntry
```

(5) *Property 1097*: Prove for correct PC set when returned from debug mode.

```
write_wb_pipe_reg && debug_return
|-> wb_InstPC == csrDpc
```

(6) *Property 1098*: Prove for correct PC set when previous instruction was *jal* instruction.

```
!firstInstruction && write_wb_pipe_reg
&& last_jal |-> wb_InstPC == jal_nextPC
```

(7) *Property 1099*: Prove for correct PC set when previous instruction was *jalr* instruction.

```
!firstInstruction && write_wb_pipe_reg
&& last_jalr |-> wb_InstPC == jalr_nextPC
```

(8) *Property 1100*: Prove for correct PC set when previous instruction branch was taken.

```
!firstInstruction && write_wb_pipe_reg
&& last_branch |-> wb_InstPC == branch_nextPC
```

(9) *Property 1101*: Prove for correct PC set when previous instruction was *MRET*.

```
!firstInstruction && write_wb_pipe_reg
&& last_mret |-> wb_InstPC == mret_nextPC
```

4.8 Register Update

Register acts as a temporary storage of instruction, data or address. Some registers serve for a special own purpose in RISC-V, for instance register x0 is fixed to 0, register x1 holds the return address for a call and register x2 is a stack pointer. Other registers are available for general purpose usage such as function arguments, saved register or temporaries. Any alteration of the register value could lead to undesired functionality change or even denial of service, e.g. if stack pointer maliciously modified. Hence, we prove the following properties for register updates.

(1) *Property 1102*: The register update as the result of executing an instruction is the register specified as the target register by the instruction.

```
WbWriteReg |-> RegBankWbEn &&
(RegBankWbRd[4:0] == InstCode[11:7]);
```

(2) *Property 1103-1134*: Register change implies that it is the instruction target (example for GPR 1).

```
true ##1 RegData1[31:0] != $past(RegData1[31:0])
|-> $past(InstCode[11:7] == 5'd1);
```

4.9 Memory Access

Memory access in RISC-V is done through the load and store instructions. Attackers often target these instructions to modify the values transferred to memory by load or store operations. We formally verified the properties below for correctness of the load and store instruction.

(1) *Property 1135-1140*: In memory stores, the value sent to the memory subsystem is exactly the value of the register specified in the store instruction. Our properties prove that the rightmost bits of the register are sent to Load Store Unit according to the target address.

```
(dataoffset_lsuBusReqAddr == 2'h01 && InstValid
&& Opcode == Store) |-> (BusReqWtData ==
{RegData[InstCode[rs2]][23:0], 8'h00});
```

(2) *Property 1141-1145*: In memory loads, the value stored in the target register is exactly the value from the memory subsystem.

(3) *Property 1146*: The address sent to the memory subsystem is exactly the effective address given the GPR values and instruction contents (i.e., addressing mode and immediate).

```
(Opcode == Store && (Funct3 == SW||SH||SB) ||
(Opcode==Load && (Funct3==LW||LH||LB||LHU||LBU)))
|-> (BusReqAddr == (RegData[rs1] + Immediate));
```

5 RESULTS

We identified and formally verified 1146 properties that are grouped under 9 categories as shown in Table 4 using Jasper formal verification tool. Some design bugs have been discovered during our formal verification experiments with Jasper. A counterexample is provided for each violation trace for debugging purpose by Jasper. The property violations will be explained and discussed below. Runtime for these verification experiments ranges between 0.01s to 4000s depending on complexity of the property, besides the Control Flow properties. For the Control Flow properties, all properties besides the properties for the standard PC increment (*Property 1094*) and the property for the *MRET* (*Property 1101*) instruction could be verified successfully within 24 hours. The two properties mentioned could not be verified successfully in less than 48 hours. We applied blackboxing of some submodules of the processor for verification of the standard PC increment and the *MRET* property and with this, those properties could also be proven successfully within 24 hours.

Table 4: Formal Verification Results

Categories	Properties		
	SVA	***Pass***	***Fail***
Instruction Execution	10	10	0
Control and Status Register	394	280	114
Debug Operation	14	14	0
Exception, Interrupt and Trap Handling	87	87	0
Mode Transition	13	13	0
Physical Memory Protection	574	574	0
Control Flow	9	8	1
Register Update	33	33	0
Memory Access	12	12	0
Total	1146	1031	115

5.1 Counterexample 1

For *Property 229-285*, we have proven noncompliance with specification. According to the RISC-V specification [4], if `rd=x0`, then the instruction CSRRW or CSRRWI shall not read and shall not cause any of the side effects that might occur on a CSRRW or CSRRWI read. However, current NS31A allows the read access to the CSRRW or CSRRWI even if `rd=x0`. This is not consistent to the RISC-V Specification, and is considered as erratum.

5.2 Counterexample 2

For *Property 286-342*, we have proven noncompliance with specification. According to the RISC-V specification [4], if `rs1=x0` or `uimm=0`, then the instruction CSRRS/C or CSRRS/CI shall not write and shall not cause any of the side effects that might occur on a CSRRS/C or CSRRS/CI write. However, current NS31A allows the write access to the CSRRS/C or CSRRS/CI even if `rs1=x0` or `uimm=0`. This is not consistent to the RISC-V Specification, and is considered as erratum.

5.3 Counterexample 3

For *Property 1101* on the PC verification, we discover a bug that occurs when a *MRET* instruction is in the *WB* stage and additionally an exception appears, while the CPU is in debug mode. As the processor is in debug mode, the next PC should here not be the PC for the exception, but the successor PC is decided by the *MRET* instruction. Nevertheless, we found a scenario where the PC was the PC of the exception handler.

6 CONCLUSION

As conclusion, in our work we identified a comprehensive set of security-critical properties within the specification and microarchitecture in a RISC-V processor and formally verified them using formal verification. We grouped our security-critical properties in 9 groups and elaborated them in detail. Our set of security-critical properties and their formal verification enables to increase the security-level of RISC-V processors considerably, which is a very important work due to the growing dispersion of RISC-V processors. The verification results also show that formal verification scales well for most of the properties. For a few properties, we had to apply additional abstraction techniques like blackboxing, to obtain proofs. For future work, we intend to extend our verification on security-critical properties to formal verification of absence for Hardware Trojans. Additionally, we intend to research the usability of our properties as runtime monitors to increase security guarantees during processor operation.

ACKNOWLEDGMENTS

This work was partially funded by the Federal Ministry of Education and Research (BMBF) under the DIVA-IC project with funding code FKZ:16ME0279

REFERENCES

[1] 2017. *Common Criteria for Information Technology Security Evaluation, Version 3.1, revision 5.* Technical Report. Common Criteria.

[2] 2018. IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language. *IEEE Std 1800-2017 (Revision of IEEE Std 1800-2012)* (Feb 2018), 1–1315. https://doi.org/10.1109/IEEESTD.2018.8299595

[3] 2020. *NSITEXE NS31A Technical Reference Manual, Document Version NRV31210000-00-E Revision 1.0.2.* Technical Report. NSITEXE, Inc.

[4] Waterman Andrew and Asanović Krste. 2019. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 20191213.* Technical Report. RISC-V Foundation.

[5] Waterman Andrew, Asanović Krste, and Hauser John. 2021. *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 20211203.* Technical Report. RISC-V International.

[6] Padmaja Bhamidipati, Shanmukha Murali Achyutha, and Ranga Vemuri. 2021. Security Analysis of a System-on-Chip Using Assertion-Based Verification. In *2021 IEEE International Midwest Symposium on Circuits and Systems (MWSCAS).* 826–831. https://doi.org/10.1109/MWSCAS47672.2021.9531916

[7] Michael Bilzor, Ted Huffmire, Cynthia Irvine, and Tim Levin. 2011. Security Checkers: Detecting processor malicious inclusions at runtime. In *2011 IEEE International Symposium on Hardware-Oriented Security and Trust.* 34–39. https://doi.org/10.1109/HST.2011.5954992

[8] Michael Bilzor, Ted Huffmire, Cynthia Irvine, and Tim Levin. 2012. Evaluating security requirements in a general-purpose processor by combining assertion checkers with code coverage. In *2012 IEEE International Symposium on Hardware-Oriented Security and Trust.* 49–54. https://doi.org/10.1109/HST.2012.6224318

[9] M. Blum and H. Wasserman. 1996. Reflections on the Pentium division bug. *IEEE Trans. Comput.* 45, 4 (April 1996), 385–393. https://doi.org/10.1109/12.494097

[10] Kevin Cheang, Cameron Rasmussen, Dayeol Lee, David W. Kohlbrenner, Krste Asanović, and Sanjit A. Seshia. 2022. Verifying RISC-V Physical Memory Protection. arXiv:2211.02179 [cs.CR]

[11] Federal Office for Information Security. [n. d.]. *Evaluation Assurance Level (EAL).* https://www.bsi.bund.de/EN/Themen/Unternehmen-und-Organisationen/Standards-und-Zertifizierung/Zertifizierung-und-Anerkennung/Zertifizierung-von-Produkten/Zertifizierung-nach-CC/IT-Sicherheiskriterien/CommonCriteria_v31/eal_stufe.html

[12] Tora Fridholm. 2023. *Codasip and IAR Demonstrate Dual-Core Lockstep for RISC-V.* codasip.com/press-release/2023/03/14/codasip-and-iar-demonstrate-dual-core-lockstep-for-risc-v/

[13] Matthew Hicks, Cynthia Sturton, Samuel T. King, and Jonathan M. Smith. 2015. SPECS: A Lightweight Runtime Mechanism for Protecting Software from Security-Critical Processor Bugs. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems* (Istanbul, Turkey) *(ASPLOS '15).* Association for Computing Machinery, New York, NY, USA, 517–529. https://doi.org/10.1145/2694344.2694366

[14] Mark D. Hill, Jon Masters, Parthasarathy Ranganathan, Paul Turner, and John L. Hennessy. 2019. On the Spectre and Meltdown Processor Security Vulnerabilities. *IEEE Micro* 39, 2 (March 2019), 9–19. https://doi.org/10.1109/MM.2019.2897677

[15] Binod Kumar, Akshay Kumar Jaiswal, V S Vineesh, and Rushikesh Shinde. 2020. Analyzing Hardware Security Properties of Processors through Model Checking. In *2020 33rd International Conference on VLSI Design and 2020 19th International Conference on Embedded Systems (VLSID).* 107–112. https://doi.org/10.1109/VLSID49098.2020.00036

[16] Inc NSITEXE. [n. d.]. *NS31A : RISC-V 32bit CPU Which Supports ISO26262 ASIL D.* www.nsitexe.com/en/ip-solutions/ns-series/ns31a/

[17] Jasper Formal Verification Platform. [n. d.]. *Jasper RTL Apps | Cadence - Cadence Design Systems.* https://www.cadence.com/en_US/home/tools/system-design-and-verification/formal-and-static-verification/jasper-gold-verification-platform.html

[18] Alastair David Reid, Rick Chen, Anastasios Deligiannis, David Gilday, David Hoyes, Will Keen, Ashan Pathirane, Owen Shepherd, Peter Vrabel, and Ali Mustafa Zaidi. 2016. End-to-End Verification of ARM ® Processors with ISA-Formal. https://api.semanticscholar.org/CorpusID:45478983

[19] Renesas. 2023. *Renesas Expands RISC-V Embedded Processing Portfolio with New Voice-Control ASSP Solution.* www.renesas.com/br/en/about/press-room/renesas-expands-risc-v-embedded-processing-portfolio-new-voice-control-assp-solution

[20] Newsome Time and Wachs Megan. 2022. *The RISC-V External Debug Support Version 0.13.2.* Technical Report. RISC-V International.

[21] Clifford Wolf. 2017. End-to-end formal ISA verification of RISC-V processors with riscv-formal. 7th RISC-V Workshop Proceedings.

[22] Rui Zhang, Natalie Stanley, Christopher Griggs, Andrew Chi, and Cynthia Sturton. 2017. Identifying Security Critical Properties for the Dynamic Verification of a Processor. *SIGARCH Comput. Archit. News* 45, 1 (apr 2017), 541–554. https://doi.org/10.1145/3093337.3037734